Linguagem de manipulação de dados (Avançada)

Linguagem de Manipulação de Dados (DML)

*A linguagem de manipulação de dados (DML) é usada para **consultar** e **modificar** dados dentro das tabelas de um banco de dados.*

Os principais comandos DML são:

- **SELECT**: Recupera dados de uma tabela.
- **INSERT**: Insere novos dados em uma tabela.
- **UPDATE**: Modifica dados existentes em uma tabela.
- **DELETE**: Remove dados de uma tabela.

Funções de Agregação

*Funções de agregação realizam cálculos sobre um **conjunto de valores** e retornam um único valor.*

- **COUNT()**: Retorna o número de linhas.
- **SUM()**: Retorna a soma dos valores.
- **AVG()**: Retorna a média dos valores.
- **MAX()**: Retorna o valor máximo.
- **MIN()**: Retorna o valor mínimo.

SELECT Avançado

- **EXEMPLO -** *Retornar o número de alunos, a média de idade, a idade máxima e mínima por curso.*

Sintaxe Básica:

comandos.sql

```sql
SELECT curso, COUNT(*), AVG(idade), MAX(idade), MIN(idade)
FROM Alunos
GROUP BY curso;
```

SELECT Avançado

- **EXEMPLO -** *Retornar o número de alunos, a média de idade, a idade máxima e mínima por curso.*

Sintaxe Básica:

comandos.sql

```sql
SELECT curso, COUNT(*), AVG(idade), MAX(idade), MIN(idade)
FROM Alunos
GROUP BY curso;
```

curso	count	avg	max	min
Engenharia de Software	2	24,0000000000000000	30	18
Letras	1	18,0000000000000000	18	18
Direito	1	22,0000000000000000	22	22

SELECT Avançado

- **SUBCONSULTAS**

Subconsultas são consultas aninhadas dentro de outra consulta.

Sintaxe Básica:

comandos.sql

```sql
SELECT nome
FROM Alunos
WHERE idade > (SELECT AVG(idade) FROM Alunos);
```

SELECT Avançado

- **SUBCONSULTAS**

comandos.sql

```sql
SELECT nome
FROM Alunos
WHERE idade > (SELECT AVG(idade) FROM Alunos);
```

1	Willian	30	Engenharia de Software
2	João	18	Engenharia de Software
3	Maria	22	Direito
4	José	18	Letras

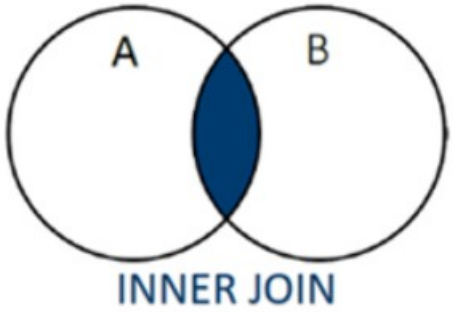
nome
Willian

SELECT Avançado

- **JOINS**
  - ***JOIN*** *é utilizado para* ***combinar informações*** *de duas ou mais tabelas*
  - Existem tipos diferentes de JOINs
    - INNER JOIN
    - LEFT JOIN
    - RIGHT JOIN
    - FULL JOIN

- **INNER JOIN**

  - *O **INNER JOIN** retorna apenas as linhas que têm correspondência em ambas as tabelas envolvidas na junção.*
  - ***Combina linhas de duas tabelas** onde há uma correspondência nos valores das colunas especificadas.*



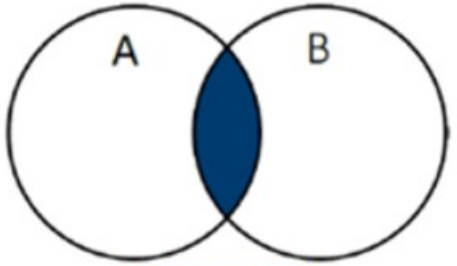
- **EXEMPLOS**

Alunos

id	nome	idade	curso_id
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103
1	Ana	21	101
5	Willian	31	(Null)

Cursos

id	nome
101	Engenharia de Software
102	Ciência da Computação
103	Administração



INNER JOIN

- **EXEMPLOS**

Alunos

id	nome	idade	curso_id
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103
1	Ana	21	101
5	Willian	31	(Null)

Cursos

id	nome
101	Engenharia de Software
102	Ciência da Computação
103	Administração

Como retornar apenas os alunos que estão matriculados em cursos que têm correspondência na tabela Cursos?

SELECT Avançado

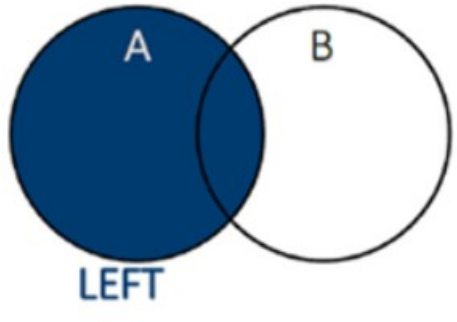
- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
INNER JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software

- **LEFT JOIN**
  - *O **LEFT JOIN** (ou **LEFT OUTER JOIN**) retorna todas as linhas da tabela **à esquerda** e as **correspondentes** da **tabela à direita**.*
  - *Para as linhas da tabela à esquerda que não têm correspondência na tabela à direita, os valores das colunas da tabela à direita serão NULL.*



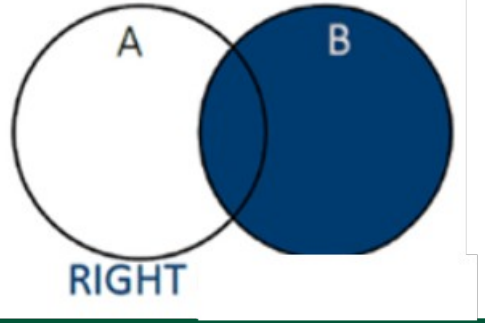
- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
LEFT JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software
Willian	(Null)

- **RIGHT JOIN**
  - *O **RIGHT JOIN** (ou **RIGHT OUTER JOIN**) retorna todas as linhas da **tabela à direita** e as correspondentes da **tabela à esquerda**.*
  - *Para as linhas da tabela à direita que não têm correspondência na tabela à esquerda, os valores das colunas da tabela à esquerda serão NULL.*



- **EXEMPLOS**

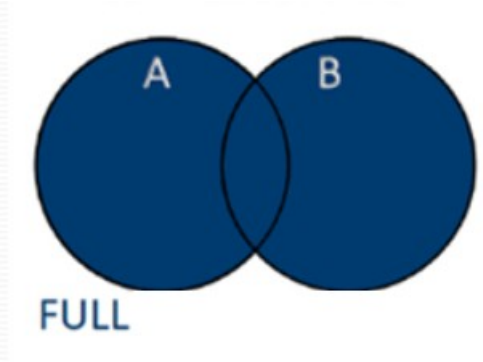
comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
RIGHT JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software
(Null)	Direito

- **FULL JOIN**

  - *O **FULL JOIN** (ou **FULL OUTER JOIN**) **retorna todas as linhas** quando há uma correspondência em uma das tabelas.*
  - *Retorna NULL para linhas sem correspondência em ambas as tabelas.*



SELECT Avançado

- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome, cursos.nome AS curso_nome
FROM alunos
FULL JOIN cursos ON alunos.curso_id = cursos.id;
```

nome	curso_nome
Bruno	Ciência da Computação
Carlos	Engenharia de Software
Daniela	Administração
Ana	Engenharia de Software
Willian	(Null)
(Null)	Direito

- **Exemplo com Subconsulta e INNER JOIN**

  - Suponha que temos as tabelas **Alunos**, **Cursos**, e **Professores**.

  - Queremos encontrar os **nomes dos alunos que estão matriculados em cursos ministrados por um professor específico**.

SELECT Avançado

- **EXEMPLOS**

Alunos

id	nome	idade	curso_id
2	Bruno	22	102
3	Carlos	23	101
4	Daniela	20	103
1	Ana	21	101
5	Willian	31	(Null)

Cursos

id	nome	professor_id
104	Direito	(Null)
101	Engenharia de Software	201
102	Ciência da Computação	202
103	Administração	203

Professores

id	nome
201	Prof. A
202	Prof. B
203	Prof. C

SELECT Avançado

- **EXEMPLOS**

comandos.sql

```sql
SELECT alunos.nome
FROM alunos
WHERE alunos.curso_id IN (
    SELECT cursos.id
    FROM cursos
    INNER JOIN professores ON cursos.professor_id = professores.id
    WHERE professores.nome = 'Prof. A'
);
```

nome
Carlos
Ana